

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..15

ENDPOINT MIDDLEWARE

 Copy as Markdown

# Endpoint middleware

Middleware wraps every request to the API server, before and after your endpoints, making it suitable for logging, caching, and rate limiting. Serverpod middleware follows the middleware interface of [Relic](#), the HTTP server framework Serverpod is built on. To add middleware to web server routes instead, scoped to path prefixes, see [Web server middleware](#).

## Add middleware to your server

Add middleware to your server in the `run` function before starting the server:

```
import 'src/generated/serverpod.dart';

void run(List<String> args) async {
  final pod = Serverpod(args);

  // Add middleware before starting
  pod.server.addMiddleware(myCustomMiddleware());

  await pod.start();
}
```

Middleware runs on every API-server request in the order it is added: not only endpoint method calls, but also the health check, WebSocket upgrades for [streaming](#), and file-storage requests. Keep each middleware focused and efficient, and return early for paths it does not care about.

## Create custom middleware

### Middleware signature

A middleware is a function with this signature:



```
typedef Handler = FutureOr<Result> Function(Request request);
typedef Middleware = Handler Function(Handler innerHandler);
```

The return value is a `Result`, which can be a `Response`, `Hijack`, or `WebSocketUpgrade`. Check `isResponse` before modifying response-specific fields. The types come with the `package:serverpod/serverpod.dart` import. No separate Relic import is needed.

## Add a header to all responses

Here's a basic middleware that adds a custom header to all responses:

```
Middleware customHeaderMiddleware() {
  return (Handler innerHandler) {
    return (Request req) async {
      // Inspect or modify `req` here if needed

      // Call the inner handler to process the request
      final result = await innerHandler(req);

      // Modify the response
      if (result is Response) {
        return result.copyWith(
          headers: result.headers.transform((h) {
            h['X-Custom-Header'] = ['my-value'];
          }),
        );
      }

      return result;
    };
  };
}
```

## Rate limit requests

Middleware can answer a request itself instead of calling the inner handler, which is how a rate limiter rejects excess traffic. This example allows each client IP a fixed number of requests per time window:

```
import 'package:serverpod/serverpod.dart';

/// Limits each client IP to [maxRequests] requests per [window].
///
/// Requests over the limit receive a 429 Too Many Requests response.
Middleware rateLimitMiddleware({
  int maxRequests = 60,
  Duration window = const Duration(minutes: 1),
}) {
  final clients = <String, ({DateTime windowStart, int count})>{};

  return (Handler innerHandler) {
    return (Request req) async {
```

```

final ip = req.connectionInfo.remote.address.toString();
final now = DateTime.now();

var client = clients[ip];
if (client == null || now.difference(client.windowStart) ≥ window) {
    // Start a new window for this client.
    client = (windowStart: now, count: 0);
}
client = (windowStart: client.windowStart, count: client.count + 1);
clients[ip] = client;

if (client.count > maxRequests) {
    return Response(
        429,
        body: Body.fromString('Too many requests. Try again later.'),
    );
}

return innerHandler(req);
};
};
}

```

When adapting it, keep three things in mind:

- Behind a load balancer or proxy, `req.connectionInfo` is the nearest network hop. Read the client address from a forwarding header such as `X-Forwarded-For` instead.
- Middleware wraps all API-server requests, so the counter also ticks for health checks and other non-endpoint traffic.
- The in-memory map grows with each new client IP and is local to one server. Production quota patterns need eviction, and state shared across servers such as a database counter.

## Handle errors in middleware

Middleware can catch errors from the handlers it wraps, for example to record metrics, before rethrowing. Serverpod's own error handling wraps all user middleware, so a rethrown exception still gets the standard status mapping. There is no `Session` in middleware, so use your own logging or metrics facility here.

```

Middleware errorHandlerMiddleware() {
    return (Handler innerHandler) {
        return (Request req) async {
            try {
                return await innerHandler(req);
            } catch (e) {
                // Record metrics or logs here.

                // Re-throw to let Serverpod handle it
                rethrow;
            }
        };
    };
}

```

```
};  
}
```